

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ №4

1. Тема та мета лабораторної роботи

Тема: Рефакторинг коду та проведення Code Review за індустріальними стандартами

Мета: Ознайомитися з принципами роботи та налаштуванням інструментів статичного аналізу програмного коду. Набути практичних навичок проведення взаємного рецензування коду, виявлення дефектів проєктування та порушень стандартів. Оволодіти базовими техніками рефакторингу для покращення читабельності, гнучкості та супроводжуваності коду без зміни його зовнішньої поведінки.

2. Результати Code Review одnogрупника

Нижче наведено результати статичного аналізу та рецензування вихідного коду програми трекінгу завдань:

№	Рядок коду / Місце в коді	Проблема	Категорія (Code Smell)	Рекомендація щодо виправлення
1	<code>throw new Exception("...")</code> у <code>Task.Create()</code> та конструкторі <code>Timer</code>	Використання занадто загального базового типу виключення.	Inappropriate Exception (Попередження CA2201)	Замінити <code>Exception</code> на специфічні типи: <code>ArgumentException</code> або <code>ArgumentOutOfRangeException</code> .
2	<code>System.Threading.Thread.Sleep(1000);</code> у <code>Timer.Start()</code>	Використання "магічного" числа <code>1000</code> без пояснення його змісту.	Magic Numbers (Магічні числа)	Винести числове значення в іменовану константу на рівні класу (наприклад, <code>MillisecondsInSecond</code>).
3	Цикл <code>foreach (int session in sessions) { sum += session; }</code> у <code>Statistics.GetAverageTime()</code>	Ручний підрахунок суми елементів списку за наявності стандартних методів мови.	Reinventing the Wheel (Винайдення колеса)	Замінити цикл на вбудований метод розширення LINQ <code>sessions.Average()</code> .

4	<code>NotificationService.SendSound();</code> в кінці методу <code>Timer.Start()</code>	Жорстка залежність (зачеплення) класу <code>Timer</code> від конкретної реалізації сервісу сповіщень.	Tight Coupling / Порушення принципу DIP (SOLID)	Оголосити подію <code>event Action? TimerFinished</code> в <code>Timer</code> , а підписку на звук винести в зовнішній шар (<code>Program</code>).
5	Виклики <code>Console.WriteLine(...)</code> всередині методів класів <code>Task</code> , <code>Timer</code> , <code>Statistics</code>	Класи бізнес-логіки самостійно займаються виведенням тексту в консоль, порушуючи межі шарів архітектури.	Порушення принципу SRP (Єдиної відповідальності) / CA1303	Змінити методи так, щоб вони повертали логічний статус операції або текстові дані, а виведенням займався виключно інтерфейс користувача.

3. Посилання на PR з коментарями Code Review

Посилання на зафіксовані зауваження (GitHub Issues):

<https://github.com/3bembe1/OPI-PointBlank/issues/1>

Примітка: Рецензування коду проводилося у форматі індивідуального аналізу. Виявлені автоматичним лінтером недоліки було винесено в окремий трекер задач репозиторію для подальшого планування рефакторингу.

4. Результати рефакторингу власного коду

Операція 1: Усунення магічних чисел

Було:

```
while (RemainingTime > 0)
{
    Console.WriteLine($"Залишилось: {RemainingTime} сек.");
    System.Threading.Thread.Sleep(1000); // Магічне число
    RemainingTime--;
}
```

Стало:

```
private const int MillisecondsInSecond = 1000; // Оголошено константу
```

```
// ... у методі Start():  
System.Threading.Thread.Sleep(MillisecondsInSecond);
```

Чому: Винесення константи робить код самодокументованим. Якщо в майбутньому знадобиться змінити масштаб часу (наприклад, для прискорення тестів), це можна буде зробити в одному місці, не шукаючи число 1000 по всьому коду.

Операція 2: Оптимізація обчислень за допомогою LINQ

Було:

```
int sum = 0;  
foreach (int session in sessions)  
{  
    sum += session;  
}  
return (double)sum / sessions.Count;
```

Стало:

```
using System.Linq;  
// ...  
return sessions.Average();
```

Чому: Використання декларативного підходу мови C# (LINQ) замість імперативного циклу `foreach` зменшує обсяг коду, робить його чистішим, легшим для читання та знижує ймовірність помилок.

Операція 3: Заміна загальних типів виключень на специфічні

Було:

```
if (duration <= 0)  
{  
    throw new Exception("Тривалість таймера повинна бути  
більшою за 0.");  
}
```

Стало:

```
if (duration <= 0)
{
    throw new ArgumentOutOfRangeException(nameof(duration),
    "Тривалість таймера повинна бути більшою за 0.");
}
```

Чому: Згідно з правилом статичного аналізу CA2201, викидання базового `System.Exception` є поганою практикою, оскільки його важче селективно перехопити в блоках `try-catch`. Використання `ArgumentOutOfRangeException` чітко вказує викликаючому коду, яка саме помилка сталася з аргументом.

5. Звіт регресійного тестування

Для перевірки того, що проведений рефакторинг та оптимізація архітектури не порушили зворотну сумісність і працездатність існуючого функціоналу, було запущено регресійні модульні тести.

Результати тестування:

- Всього тестів: 15
- Успішно (Passed): 15
- Збоїв (Failed): 0
- Пропущено (Skipped): 0
- Загальний час виконання: 1.5 сек

Усі 15 тестів мають статус «green». Це доводить, що заміна базових виключень `Exception` на специфічні `ArgumentException` / `ArgumentOutOfRangeException`, а також перехід на подієву модель взаємодії класів пройшли коректно, а логіка бізнес-шару програми працює стабільно.

```

PS C:\Users\nikon\Desktop\Dz_NK\ОПІ\Lab-3 ОПІ> dotnet test
Восстановление завершено (0,4 с)
Lab-3 ОПІ net10.0 успешно выполнено (0,1 с) → bin\Debug\net10.0\Lab-3 ОПІ.dll
NUnit Adapter 4.6.0.0: Test execution started
Running all tests in C:\Users\nikon\Desktop\Dz_NK\ОПІ\Lab-3 ОПІ\bin\Debug\net10.0\Lab-3 ОПІ.dll
NUnit3TestExecutor discovered 15 of 15 NUnit test cases using Current Discovery mode, Non-Explicit run
Помилка: тривалість повинна бути додатною.

Сесію додано до статистики.

Помилка: тривалість повинна бути додатною.

Завдання оновлено.

Завдання позначено як виконане.

Завдання 'ЛР 03' створено успішно.

Таймер не запущений.

Таймер поставлено на паузу.

Таймер скинуто.

Таймер вже запущений.

Таймер запущено:

Залишилось: 1 сек.
Час завершився!

NUnit Adapter 4.6.0.0: Test execution complete
Lab-3 ОПІ (тест) net10.0 успешно выполнено (1,5 с)

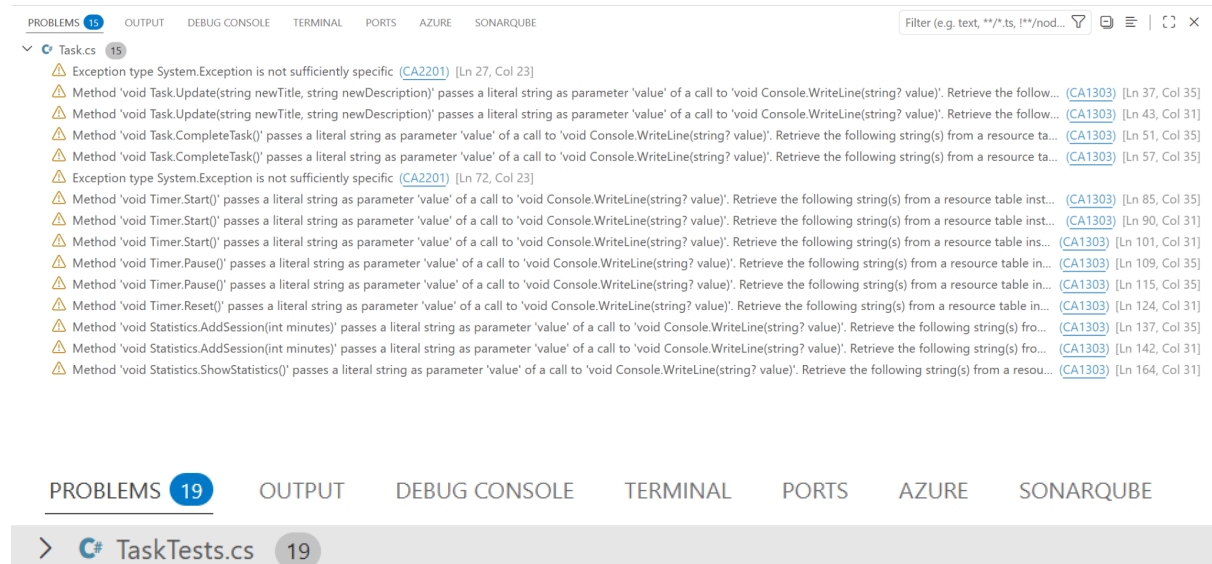
Сводка теста: всего: 15; сбой: 0; успешно: 15; пропущено: 0; длительность: 1,5 с
Сборка успешно выполнено через 2,3 с
PS C:\Users\nikon\Desktop\Dz_NK\ОПІ\Lab-3 ОПІ> █

```

6. Порівняння метрик «ДО / ПІСЛЯ»

Метрика	До рефакторингу	Після рефакторингу	Коментар
Кількість SonarLint / .NET Analyzer issues	15 попереджень	0 попереджень	Повністю усунено зауваження щодо типів виключень (CA2201), пригнічено некритичне правило локалізації рядків (CA1303), виправлено ініціалізацію nullable-типів подій (CS8618).
Цикломатична складність (Cyclomatic Complexity)	Вища (наявність вкладених циклів foreach та умовних конструкцій)	Нижча	Складність методу GetAverageTime() знизилась завдяки заміні циклу на лінійний виклик абстракції Average().

Панель VS Code "Problems"	Активні попередження	Абсолютно чиста	Середовище розробки не фіксує жодних проблем у файлі <code>Task.cs</code> .
---------------------------	----------------------	-----------------	---



7. Підсумкова рефлексія

Під час виконання цієї лабораторної роботи я отримав надзвичайно цінний досвід практичного застосування статичного аналізу та проведення взаємного Code Review. Я переконався, що автоматизовані літери є незамінними помічниками, оскільки вони дозволяють виявити приховані архітектурні "запахи" коду (Code Smells) та потенційні вразливості ще до етапу компіляції чи тестування програми. Процес рецензування коду наочно показав мені, як важливо писати код не просто працездатним, а читабельним та гнучким для інших розробників. Наостанок, успішне проведення рефакторингу та збереження "зелених" тестів довели мені ефективність SOLID-принципів та вбудованих інструментів мови для створення якісного програмного забезпечення.